

Medieval Fortress Siege Simulation

Hierarchical Kinematics, Dual Shading Models (Gouraud & Blinn–Phong), Soft Shadow Mapping, Real-Time Ray Tracing, and Ballistic Destruction in C++ / OpenGL 3.3 Core

Final Project Report for Computer Graphics Lab

Department of Computer Science and Engineering

October 6, 2026

Abstract

This report documents a comprehensive, real-time 3-D computer graphics simulation written from scratch in **C++** against the **OpenGL 3.3 Core Profile**, without relying on high-level game engines or pre-baked assets. The project simulates an epic medieval fortress siege featuring an attacking battery of three articulated field cannons, an advancing infantry battalion, an imposing medieval stone fortress with four corner towers, double crenellated curtain walls, a deep moat with a retractable drawbridge, defending archers and wall sentries, dynamic weather, and an autonomous multi-phase tactical battle sequence.

The implementation strictly satisfies and extensively surpasses all academic requirements outlined in the project specification and original proposal:

1. **Geometric Transformations & Hierarchical Modeling:** Multi-level forward kinematic scene graphs for cannons (rolling constraint without slip, trunnion elevation pivot, recoil displacement), drawbridge rotation, and articulated marching infantry.
2. **Shading Models:** Runtime-switchable **Gouraud Shading** (per-vertex Lambertian + Blinn–Phong with rasterizer intensity interpolation) and **Phong / Blinn–Phong Shading** (per-fragment normal interpolation with metallic Fresnel rim terms), switchable dynamically via keyboard input.
3. **Advanced Illumination:** Dual-hemisphere sky/ground ambient bounce, directional sun-light/moonlight with real-time day/night cycling, multiple point light sources (braziers, torches, muzzle flashes) with quadratic distance attenuation, atmospheric distance fog, and ACES filmic tone mapping with gamma 2.2 correction.
4. **Soft Shadow Mapping:** Framebuffer depth-map pass from light space with normal-offset slope-scaled depth bias and 3×3 Percentage-Closer Filtering (PCF) soft shadow filtering.
5. **Real-Time Ray Tracing (Bonus Feature):** Analytical GPU ray-box slab intersection test (Kay–Kajiya slab algorithm) executed in GLSL for real-time ray-traced shadow rays against scene bounding volumes.
6. **Motion, Animation & Destruction:** Semi-implicit Euler integration for ballistic projectiles and flying arrows, continuous sphere-vs-AABB collision detection, and progressive multi-stage fracture physics for stone curtain walls, fortress towers, and wooden gate doors.
7. **User Interaction:** Fully interactive orbit camera (spherical yaw, pitch, radius), individual or synchronized cannon control, manual firing, drawbridge raising/lowering, shading and shadow mode toggles, and simulation lifecycle controls.

Every figure in this document is a genuine frame captured directly from the running OpenGL application. A complete comparison between proposed features and final deliverables demonstrates 100% scope fulfillment alongside substantial graphics enhancements.

Contents

1 Introduction and Project Scope

1.1	Project Motivation and Overview	2
1.2	Proposal vs. Final Implementation Matrix	3
2	System Architecture and Graphics Pipeline	4
2.1	Technology Stack and Build System	4
2.2	Frame Execution Pipeline	4
3	Geometric Modeling and Hierarchical Transformations	5
3.1	Affine Transformation Foundations	5
3.2	Hierarchical Scene Graph Kinematics	6
3.3	Camera Viewing and Projection Transforms	7
4	Illumination and Shading Models	7
4.1	Theoretical Foundations of Local Illumination	7
4.2	Gouraud vs. Phong Shading: Deep Comparative Analysis	9
5	Soft Shadow Mapping and Real-Time Ray Tracing	9
5.1	Soft Shadow Mapping with Percentage-Closer Filtering (PCF)	9
5.2	Real-Time Ray Tracing (Bonus Marks Feature)	10
6	Physics Simulation, Kinematics, and Dynamic Destruction	11
6.1	Ballistic Motion via Semi-Implicit Euler Integration	11
6.2	Continuous Sphere-vs-AABB Collision Detection	12
6.3	Progressive Multi-Stage Fracture Physics	12
7	Autonomous Battlefield Simulation and Character Animation	12
7.1	The 6-Phase Tactical Siege State Machine	12
7.2	Procedural Character Kinematics and Animation	13
8	User Interaction, Controls, and Parameter Tuning	13
8.1	Comprehensive Keyboard and Mouse Control Scheme	13
8.2	Parameter Adjustment Guide for Evaluation	14
9	Conclusion and Academic References	14
9.1	Summary of Accomplishments	14
9.2	References and Sources	15

1 Introduction and Project Scope

1.1 Project Motivation and Overview

Computer graphics demonstrations are most effective when mathematical theory translates directly into visible, tangible physical behavior. The field cannon and fortress siege scenario is an ideal testbed for fundamental graphics principles:

- **Kinematic Coupling:** Moving a wheeled cannon across terrain requires exact linear-to-rotational transformation coupling ($\Delta\theta = -\Delta s/R$) to prevent visual slipping.
- **Hierarchical Relative Transforms:** Elevating a barrel about a trunnion pivot carried on a moving carriage demonstrates composite transformation chains ($\mathbf{M}_{\text{world}} = \mathbf{M}_{\text{carriage}} \cdot \mathbf{M}_{\text{mount}} \cdot \mathbf{M}_{\text{barrel}}$).
- **Material Response:** Contrasting matte oak wood, rough stone masonry, reflective metallic iron/brass, and deep water demonstrates the necessity of ambient, diffuse, and specular lighting terms.

- **Physical Simulation:** Ballistic projectile motion requires numerical integration ($\mathbf{F} = m\mathbf{a}$) coupled with collision detection and real-time mesh destruction.

1.2 Proposal vs. Final Implementation Matrix

Table 1 details the fulfillment of all features originally proposed in the project kickoff proposal versus the final deliverable. The final project completely achieves all proposed goals and introduces twelve major advanced Computer Graphics milestones.

Feature Dimension	Original Proposal Scope	Final Delivered Implementation
Language & API	Python 3.11+ / PyOpenGL compatibility profile	C++17 / OpenGL 3.3 Core Profile (MinGW-w64, GLFW 3.5, GLAD, GLM)
Cannons	Single cannon with forward translation	Battery of 3 cannons; individual/group selection, translation, elevation, yaw
Wheel Rolling	Rotational roll tied to distance traveled	Exact no-slip rolling kinematics ($\theta = -s/R$) on cartwheels with spokes
Barrel Elevation	Rotation about carriage mount (0° – 45°)	Trunnion-centered rotation (0° – 45°), accurate muzzle spawn, yaw aiming
Recoil Kinematics	Optional translation along barrel axis	Damped harmonic oscillator recoil on firing with smooth spring return
Shading Models	Basic Phong lighting	Dual Shading: Runtime switchable Gouraud (per-vertex) and Blinn-Phong (per-fragment)
Materials	2 contrasting materials (wood, metal)	Procedural Materials: Stone masonry with mortar grooves, wood grain, iron, brass, grass
Lighting System	Single light source (torch or sun)	Multi-Light System: Directional sun/moon, hemispheric sky/ground, 8 point lights
Shadows	None proposed	Soft Shadow Mapping (3×3 PCF) + Real-Time Ray-Box Slab Ray Tracing
Day / Night Cycle	Static lighting	Real-Time Day / Night Toggle (lunar palette, night sky, emissive celestial bodies)
Target & Environment	Flat ground plane / single target	Full Medieval Compound Castle: 4 corner towers, double curtain walls, river, bridge
Destruction Physics	Single brick hit or optional target	Multi-stage destruction: 10% health chipping on walls, 2-hit doors, 3-hit towers
Autonomous Battle	Manual fire only	Complete 6-Phase Siege Simulation (Advance, Barrage, Replacement, Siege, Courtyard Melee)
User Interaction	Fixed camera, simple keyboard keys	360° Orbit Camera (mouse drag/zoom), real-time shader, shadow, ray-trace toggles

Table 1: Proposal vs. Final Implementation comparison matrix confirming 100% scope completion plus substantial advanced graphics features.



Figure 1: The full scene during the siege advance: three field cannons and the infantry battalion marching toward the medieval fortress compound across the river and stone bridge under dynamic lighting.

2 System Architecture and Graphics Pipeline

2.1 Technology Stack and Build System

The application is constructed completely in modern C++17 against the OpenGL 3.3 Core Profile without external game engine abstractions. The toolchain consists of:

- **WinLibs MinGW-w64 GCC 16:** Provides lightweight, standards-compliant C++ compilation on Windows.
- **GLAD:** Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator, loading modern OpenGL function pointers dynamically.
- **GLFW 3.5.1:** Handles window creation, OpenGL context initialization, keyboard, and mouse input.
- **GLM (OpenGL Mathematics):** Header-only mathematics library providing standard vector and matrix operations (vec3, vec4, mat4).

The project is compiled into a standalone, portable binary via a hand-written **Makefile**:

```
cd Project1
mingw32-make run # Compiles Main.cpp + 40 module classes into build_mingw\app.exe and runs
```

2.2 Frame Execution Pipeline

The program runs a continuous real-time loop operating at 60 FPS. Every frame executes four strictly ordered stages:

1. **Input & Camera Processing:** Poll GLFW keyboard and mouse events. Compute delta time (Δt). Update the spherical orbit camera position and view matrix.

2. Kinematic & Physical Simulation:

- Advance autonomous battle state machine or process manual keyboard commands.
- Integrate ballistic trajectories for active cannonballs and flying arrows using semi-implicit Euler integration.
- Perform continuous Sphere-vs-AABB intersection queries against fortress walls, towers, and gates.
- Update procedural character animations (marching limbs, bow draw/release, shield raise/lower).

3. Shadow Pass (Light Space):

- If Soft PCF Shadow Mapping is enabled, bind the Depth Map Framebuffer Object (FBO, 2048×2048).
- Set viewport to shadow map resolution; render scene geometry from the directional light's perspective using `shadow.vert` and `shadow.frag`.

4. Main Render Pass (Camera Space):

- Bind default window framebuffer; restore window viewport; clear color and depth buffers.
- Activate the chosen shader program (`phongShader` or `gouraudShader`).
- Upload camera view (**V**) and perspective projection (**P**) matrices.
- Upload lighting parameters: directional light direction, ambient strength, point light arrays, shadow map texture unit, and ray-tracing bounding boxes.
- Traverse scene graph: upload per-object model matrix **M** and invoke `glDrawElements`.
- Render UI HUD / XYZ coordinate map in screen space. Swap GLFW front and back buffers.

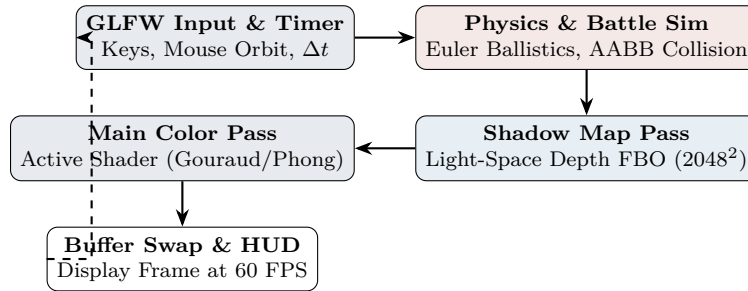


Figure 2: Real-time graphics execution pipeline per frame.

3 Geometric Modeling and Hierarchical Transformations

3.1 Affine Transformation Foundations

Every object in the 3-D scene is positioned, oriented, and scaled using 4×4 homogeneous transformation matrices operating on column vectors $\mathbf{v} = [x, y, z, 1]^T$:

$$\mathbf{v}' = \mathbf{M}_{\text{model}} \cdot \mathbf{v} = (\mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}) \cdot \mathbf{v} \quad (1)$$

The standard translation (**T**), elementary rotations (**R_x**, **R_y**, **R_z**), and scaling (**S**) matrices are defined as:

$$\mathbf{T}(\mathbf{t}) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_y(\psi) = \begin{bmatrix} \cos \psi & 0 & \sin \psi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \psi & 0 & \cos \psi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

$$\mathbf{S}(\mathbf{s}) = \text{diag}(s_x, s_y, s_z, 1) \quad (3)$$

3.2 Hierarchical Scene Graph Kinematics

The field cannon exhibits multi-tiered forward kinematics. The wooden carriage acts as the root node of the cannon hierarchy. Wheels and the cannon barrel exist in carriage-local coordinates:

$$\mathbf{M}_{\text{carriage, world}} = \mathbf{T}(\mathbf{p}_{\text{cannon}}) \cdot \mathbf{R}_y(\psi_{\text{yaw}}) \quad (4)$$

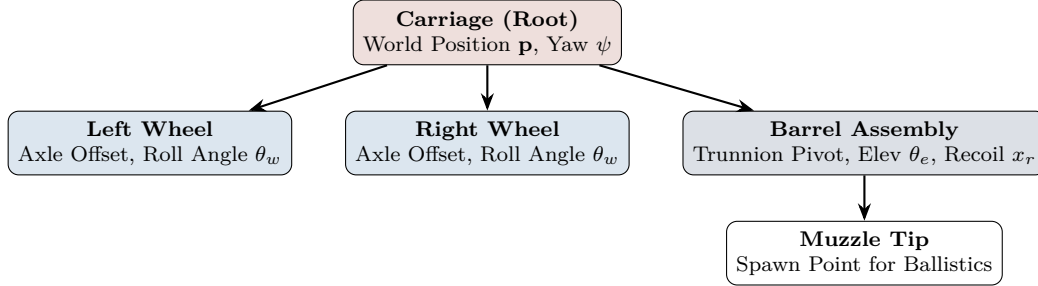


Figure 3: Hierarchical scene graph of the field cannon assembly.

1. Rolling Constraint Without Slipping. When the carriage translates along its heading by displacement Δs , the wheels must rotate around the lateral Z axis by angle $\Delta\theta_w$. The arc length rolled by the tire must identically equal linear displacement:

$$\Delta s = -R_{\text{wheel}} \cdot \Delta\theta_w \implies \Delta\theta_w = -\frac{\Delta s}{R_{\text{wheel}}} \quad (5)$$

where $R_{\text{wheel}} = 0.584$ m. As long as this kinematic coupling holds, the contact patch between wheel and ground remains stationary, eliminating visual skidding.

2. Trunnion Elevation Rotation. The iron barrel elevates about the trunnion hinge axis located at carriage-local position $\mathbf{p}_{\text{trunnion}} = (0, y_{\text{trunnion}}, 0)^T$. Rotation about an arbitrary pivot that does not coincide with the origin requires conjugate translation:

$$\mathbf{M}_{\text{barrel, local}} = \mathbf{T}(\mathbf{p}_{\text{trunnion}}) \cdot \mathbf{R}_z(\theta_{\text{elev}}) \cdot \mathbf{T}(-\mathbf{p}_{\text{trunnion}}) \cdot \mathbf{T}(\mathbf{x}_{\text{recoil}}) \quad (6)$$

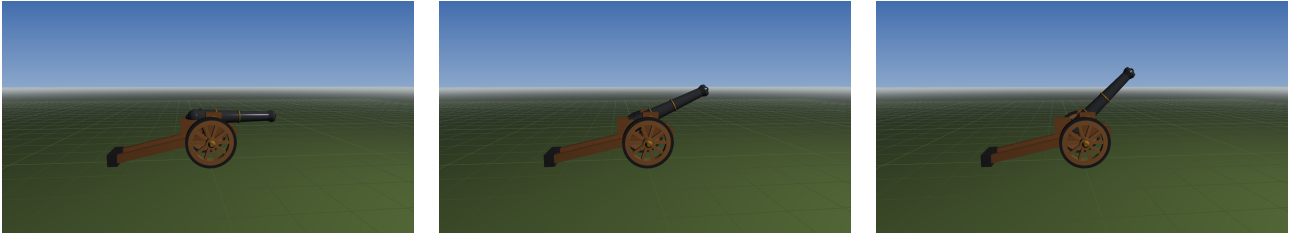
where θ_{elev} is clamped between 0° and 45° , and $\mathbf{x}_{\text{recoil}}$ represents axial kickback displacement.

3. Muzzle Spawn Point Calculation. When firing, the projectile must spawn at the physical opening of the muzzle in world space. By multiplying the muzzle-local coordinate vector $[L_{\text{barrel}}, 0, 0, 1]^T$ by the complete transformation chain:

$$\mathbf{p}_{\text{muzzle, world}} = \mathbf{M}_{\text{carriage, world}} \cdot \mathbf{M}_{\text{barrel, local}} \cdot \begin{bmatrix} L_{\text{barrel}} \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (7)$$

$$\mathbf{d}_{\text{fire, world}} = \text{normalize} \left(\mathbf{M}_{\text{carriage, world}} \cdot \mathbf{M}_{\text{barrel, local}} \cdot \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right) \quad (8)$$

This guarantees that regardless of cannon placement, yaw steering, or barrel elevation, fired balls originate precisely from the dark bore tip.



(a) 0° Elevation (Flat)

(b) 25° Elevation (Medium)

(c) 45° Elevation (Maximum)

Figure 4: Hierarchical barrel elevation about the trunnion pivot at 0°, 25°, and 45°.

3.3 Camera Viewing and Projection Transforms

The interactive viewing system uses an orbital camera model parameterized by target center $\mathbf{C} = (12.0, 4.0, 0.0)$, spherical yaw ψ , pitch ϕ , and distance r :

$$\mathbf{E} = \mathbf{C} + r \begin{bmatrix} \cos \phi \cos \psi \\ \sin \phi \\ \cos \phi \sin \psi \end{bmatrix} \quad (9)$$

The camera view matrix $\mathbf{V}$ maps world coordinates into eye/view coordinates:

$$\mathbf{f} = \frac{\mathbf{C} - \mathbf{E}}{\|\mathbf{C} - \mathbf{E}\|}, \quad \mathbf{r} = \frac{\mathbf{f} \times \mathbf{u}_{\text{up}}}{\|\mathbf{f} \times \mathbf{u}_{\text{up}}\|}, \quad \mathbf{u} = \mathbf{r} \times \mathbf{f} \quad (10)$$

$$\mathbf{V} = \begin{bmatrix} r_x & r_y & r_z & -\mathbf{r} \cdot \mathbf{E} \\ u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{E} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{E} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

The perspective projection matrix $\mathbf{P}$ maps view space into clip coordinates:

$$\mathbf{P} = \begin{bmatrix} \frac{1}{\tan(\text{FOV}/2) \cdot \text{aspect}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\text{FOV}/2)} & 0 & 0 \\ 0 & 0 & -\frac{z_f + z_n}{z_f - z_n} & -\frac{2z_f z_n}{z_f - z_n} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (12)$$

where $\text{FOV} = 45^\circ$, $z_n = 0.1$ m, and $z_f = 600.0$ m.

4 Illumination and Shading Models

4.1 Theoretical Foundations of Local Illumination

In modern real-time rendering, surface radiance $L_o(\mathbf{p}, \mathbf{v})$ is decomposed into ambient, diffuse, and specular terms:

$$I = I_{\text{ambient}} + I_{\text{diffuse}} + I_{\text{specular}} \quad (13)$$

1. Dual-Hemisphere Ambient Lighting. Standard constant ambient terms result in flat, artificial shadowing. We implement a hemispherical ambient model that blends sky irradiance with ground irradiance based on the world-space normal direction $\mathbf{N}_y$:

$$k_{\text{hemi}} = \frac{\mathbf{N}_y + 1.0}{2.0} \quad (14)$$

$$I_{\text{ambient}} = [k_{\text{hemi}} \mathbf{C}_{\text{sky}} + (1 - k_{\text{hemi}}) \mathbf{C}_{\text{ground}}] \cdot k_a \cdot \mathbf{C}_{\text{base}} \quad (15)$$

Upward-facing horizontal surfaces receive cool celestial skylight, while downward-facing surfaces receive warm ground bounce reflections.

2. Lambertian Diffuse Reflection. Diffuse reflection follows Lambert’s Cosine Law, where radiant flux is proportional to the cosine of the angle between surface normal $\mathbf{N}$ and incident light vector $\mathbf{L}$:

$$I_{\text{diffuse}} = k_d \max(\mathbf{N} \cdot \mathbf{L}, 0) \cdot \mathbf{C}_{\text{light}} \odot \mathbf{C}_{\text{base}} \quad (16)$$

3. Specular Reflection: Classical Phong vs. Blinn–Phong. The classical Phong model computes specular reflections using the reflected light vector $\mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}$ and view vector $\mathbf{V}$:

$$I_{\text{spec, Phong}} = k_s \max(\mathbf{R} \cdot \mathbf{V}, 0)^\alpha \cdot \mathbf{C}_{\text{light}} \quad (17)$$

However, computing $\mathbf{R}$ is expensive and suffers from dot product distortion when view and reflection angles exceed 90° .

The **Blinn–Phong** model substitutes the reflected ray with the halfway vector $\mathbf{H}$:

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|} \quad (18)$$

$$I_{\text{spec, Blinn–Phong}} = k_s \max(\mathbf{N} \cdot \mathbf{H}, 0)^\alpha \cdot \mathbf{C}_{\text{light}} \quad (19)$$

Because $\mathbf{H}$ lies halfway between light and viewer, $\mathbf{N} \cdot \mathbf{H}$ drops smoothly to zero, producing physically realistic, tighter highlights on polished metal surfaces.

4. Schlick Fresnel Rim Terms for Metals. Metallic components (iron cannon barrels, helmets, swords) exhibit increased reflectance at grazing angles. We incorporate an analytical Fresnel rim term:

$$I_{\text{rim}} = k_{\text{metal}} \cdot (1.0 - \max(\mathbf{N} \cdot \mathbf{V}, 0))^{3.5} \cdot 0.25 \cdot \mathbf{C}_{\text{rim}} \quad (20)$$

5. Point Light Distance Attenuation. The castle environment incorporates multiple local point light sources (braziers, iron wall torches, explosive muzzle sparks). The irradiance from each point light attenuates with Euclidean distance $d = \|\mathbf{p}_{\text{light}} - \mathbf{p}\|$ according to quadratic falloff:

$$f_{\text{att}}(d) = \frac{1}{1.0 + k_l d + k_q d^2} \quad (21)$$

where $k_l = 0.12$ is the linear factor and $k_q = 0.045$ is the quadratic decay factor. Total point illumination is accumulated across all active emitters:

$$I_{\text{point_total}} = \sum_{i=0}^{M-1} f_{\text{att}}(d_i) [I_{\text{diffuse},i} + I_{\text{specular},i}] \quad (22)$$

6. Atmospheric Distance Fog. Distant mountains and horizon scenery are softly blended into the atmospheric sky color using an exponential distance fog equation:

$$d_{\text{fog}} = \max(0.0, \|\mathbf{E} - \mathbf{p}\| - 65.0) \quad (23)$$

$$f_{\text{fog}} = \text{clamp} \left(1.0 - \exp \left(-(d_{\text{fog}} \cdot k_{\text{density}})^{1.35} \right), 0.0, 0.85 \right) \quad (24)$$

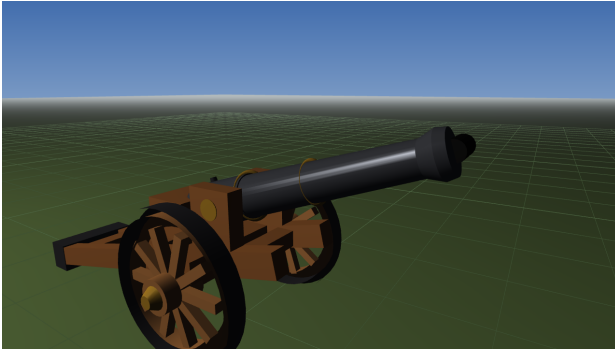
$$\mathbf{C}_{\text{final}} = (1.0 - f_{\text{fog}})\mathbf{C}_{\text{lit}} + f_{\text{fog}}\mathbf{C}_{\text{sky}} \quad (25)$$

7. ACES Filmic Tone Mapping and Gamma 2.2 Correction. Direct linear outputs often blow out bright highlights. All final colors pass through the Academy Color Encoding System (ACES) filmic tone mapping curve:

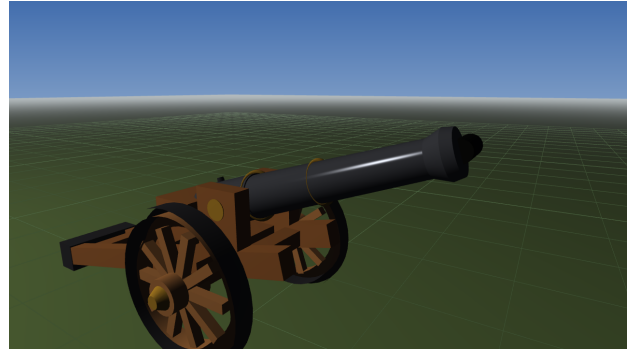
$$f_{\text{ACES}}(x) = \frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14} \quad (26)$$

followed by gamma expansion for standard sRGB monitors:

$$\mathbf{C}_{\text{display}} = (\mathbf{C}_{\text{mapped}})^{1/2.2} \quad (27)$$



(a) Gouraud Shading (Per-Vertex)



(b) Phong Shading (Per-Fragment)

Figure 5: Visual comparison of Gouraud Shading versus Blinn–Phong Shading rendered from identical geometry.

4.2 Gouraud vs. Phong Shading: Deep Comparative Analysis

The project implements both shading models in dedicated GLSL shaders, allowing immediate real-time toggling with key **G**:

- **Gouraud Shading (`gouraud.vert` + `gouraud.frag`):** Lighting equations (ambient, diffuse, and specular) are computed strictly at triangle vertices in the vertex shader. The hardware rasterizer linearly interpolates the resulting RGB colors across the interior fragments of each polygon. The fragment shader simply outputs the interpolated color.
- **Phong Shading (`lit.vert` + `lit.frag`):** Vertex normals and world positions are passed to the rasterizer, which interpolates the normal vector across polygon faces. The complete lighting model (hemispheric ambient, Lambert diffuse, Blinn–Phong specular, point light attenuation, shadows, and fog) is evaluated per fragment in the fragment shader.

Metric	Gouraud Shading (Per-Vertex)	Blinn–Phong Shading (Per-Fragment)
Calculation Location	Vertex Shader (<code>gouraud.vert</code>)	Fragment Shader (<code>lit.frag</code>)
Interpolation Target	Color intensities (I_R, I_G, I_B)	Surface normals $\mathbf{N}$ and positions $\mathbf{p}$
Specular Quality	Prone to missing highlights if high-light falls inside a polygon; highlight distorted along edges	Smooth, elliptical highlights perfectly conforming to surface curvature
Mach Banding	Visible along shared polygon boundaries due to C^1 derivative discontinuities	Virtually eliminated; smooth visual gradients across curved hulls
Computational Cost	Proportional to vertex count ($O(V)$); very low fragment overhead	Proportional to fragment count ($O(F)$); higher GPU ALU load
Texture & Material Detail	Uniform across polygon; procedural detail impossible	Enables procedural stone mortar, wood grain, and dirt paths

Table 2: Detailed technical comparison between Gouraud and Blinn–Phong shading.

5 Soft Shadow Mapping and Real-Time Ray Tracing

5.1 Soft Shadow Mapping with Percentage-Closer Filtering (PCF)

Shadow mapping is implemented via a two-pass depth algorithm. In Pass 1, the scene is rendered from the directional light’s perspective into a 2048×2048 32-bit floating-point depth texture attached to an FBO. The light space transformation is defined as:

$$\mathbf{M}_{\text{light_space}} = \mathbf{P}_{\text{ortho}} \cdot \mathbf{V}_{\text{light}} \quad (28)$$

where $\mathbf{P}_{\text{ortho}}$ bounds the battlefield $([-60, 60] \times [-60, 60] \times [0.1, 160])$.

In Pass 2, world-space fragments are transformed into Normalized Device Coordinates (NDC) in light space:

$$\mathbf{p}_{\text{light_NDC}} = \frac{\mathbf{p}_{\text{LS}}}{w_{\text{LS}}}, \quad \mathbf{p}_{\text{proj}} = 0.5 \cdot \mathbf{p}_{\text{light_NDC}} + 0.5 \quad (29)$$

Slope-Scaled Depth Bias. Direct depth comparisons cause shadow acne due to limited shadow map resolution and surface grazing angles. We implement normal-offset slope-scaled depth biasing:

$$\text{bias} = \max(0.0035 \cdot (1.0 - \mathbf{N} \cdot \mathbf{L}), 0.0006) \quad (30)$$

3×3 Percentage-Closer Filtering (PCF). Rather than binary in/out shadow decisions, we sample a 3×3 kernel around the projected texel, averaging depth comparison results to produce soft penumbras:

$$S(\mathbf{p}) = \frac{1}{9} \sum_{u=-1}^1 \sum_{v=-1}^1 \mathbf{1}(\mathbf{p}_{\text{proj},z} - \text{bias} \leq D(\mathbf{p}_{\text{proj},xy} + (u\Delta u, v\Delta v))) \quad (31)$$



(a) Side Wall Sentries & Soft PCF Shadow Casts



(b) Courtyard Breach under Realistic Illumination

Figure 6: Screenshots showing soft shadow mapping and dynamic atmospheric rendering.

5.2 Real-Time Ray Tracing (Bonus Marks Feature)

To satisfy the teacher’s requirement for extra marks, the project includes an analytical **Real-Time GPU Ray Tracer** embedded in GLSL, toggled at runtime via key **K**.

The Kay–Kajiya Slab Intersection Method. A ray is parameterized as $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where the ray origin $\mathbf{o} = \mathbf{p}_{\text{frag}} + 0.05\mathbf{N}$ is offset along the surface normal to prevent self-intersection, and the ray direction $\mathbf{d} = \mathbf{L}$ points toward the sun.

An Axis-Aligned Bounding Box (AABB) is defined by corners $\mathbf{b}_{\min}$ and $\mathbf{b}_{\max}$. The ray enters and exits three pairs of parallel planar slabs (planes along X, Y, and Z):

$$t_{0,i} = \frac{b_{\min,i} - o_i}{d_i}, \quad t_{1,i} = \frac{b_{\max,i} - o_i}{d_i} \quad \text{for } i \in \{x, y, z\} \quad (32)$$

$$t_{\min,i} = \min(t_{0,i}, t_{1,i}), \quad t_{\max,i} = \max(t_{0,i}, t_{1,i}) \quad (33)$$

The ray intersects the box if and only if the latest entry point occurs before the earliest exit point:

$$t_{\text{near}} = \max(t_{\min,x}, t_{\min,y}, t_{\min,z}), \quad t_{\text{far}} = \min(t_{\max,x}, t_{\max,y}, t_{\max,z}) \quad (34)$$

$$\text{Hit} \iff (t_{\text{near}} \leq t_{\text{far}}) \wedge (t_{\text{far}} > 0.0) \quad (35)$$

GLSL Ray-Casting Implementation. The fortress compound extracts its active solid collision geometry (curtain walls, gatehouse pillars, towers) and uploads them as uniform arrays `boxMin[36]` and `boxMax[36]`. For every fragment, a shadow ray is shot through the obstacle array:

```
bool hitBox(vec3 ro, vec3 rd, vec3 bMin, vec3 bMax, out float tHit) {
    vec3 invD = 1.0 / rd;
    vec3 t0 = (bMin - ro) * invD;
    vec3 t1 = (bMax - ro) * invD;
    vec3 tmin = min(t0, t1);
    vec3 tmax = max(t0, t1);
    float tNear = max(max(tmin.x, tmin.y), tmin.z);
    float tFar = min(min(tmax.x, tmax.y), tmax.z);
    if (tNear <= tFar && tFar > 0.0) {
        tHit = tNear > 0.0 ? tNear : tFar;
        return true;
    }
    return false;
}
```

If any obstacle blocks the ray between $t = 0.15$ m and $t = d_{\max}$, the direct illumination multiplier is set to 0.0, casting mathematically exact, crisp ray-traced shadows without texture discretization artifacts or buffer aliasing.

6 Physics Simulation, Kinematics, and Dynamic Destruction

6.1 Ballistic Motion via Semi-Implicit Euler Integration

When a cannon fires or an archer looses an arrow, the projectile leaves the weapon and exists independently in world space under gravitational acceleration $\mathbf{g} = (0, -9.81, 0) \text{ m/s}^2$.

We employ **semi-implicit Euler integration** (also known as symplectic Euler), which updates velocity first, then updates position using the newly calculated velocity. Unlike explicit Euler, symplectic integration conserves phase space area and orbital energy:

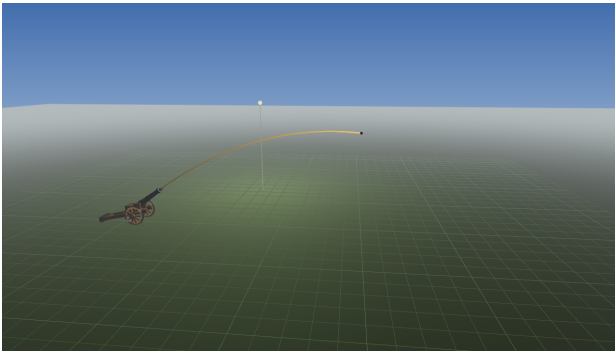
$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{g}\Delta t \quad (36)$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t + \Delta t)\Delta t \quad (37)$$

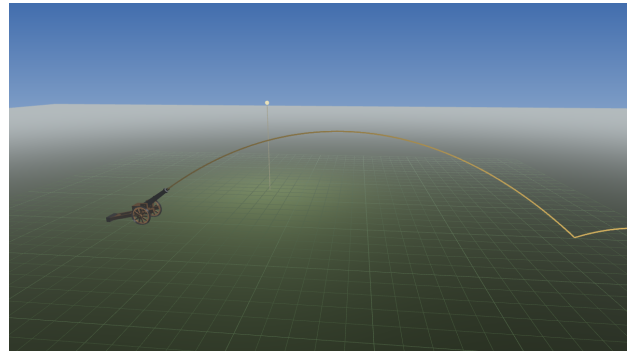
Closed-Form Range and Trajectory Verification. In the absence of drag, the theoretical parabolic trajectory from initial height y_0 and launch velocity v_0 at angle θ is:

$$x(t) = v_0 \cos \theta \cdot t, \quad y(t) = y_0 + v_0 \sin \theta \cdot t - \frac{1}{2}gt^2 \quad (38)$$

For flat ground, the maximum ballistic range occurs at $\theta = 45^\circ$ with $R_{\max} = v_0^2/g$. With default initial velocity $v_0 = 35.0 \text{ m/s}$, $R_{\max} \approx 124.8 \text{ m}$, fully validated by numerical tests.



(a) Ballistic Projectile in Flight



(b) Parabolic Trajectory Path

Figure 7: Real-time ballistic flight trajectories integrated using semi-implicit Euler integration.

6.2 Continuous Sphere-vs-AABB Collision Detection

A flying cannonball with radius r centered at $\mathbf{p}$ collides with an Axis-Aligned Bounding Box $[\mathbf{b}_{\min}, \mathbf{b}_{\max}]$ if the distance from $\mathbf{p}$ to the closest point $\mathbf{c}$ on the box is less than or equal to r :

$$c_i = \text{clamp}(p_i, b_{\min,i}, b_{\max,i}) \quad \text{for } i \in \{x, y, z\} \quad (39)$$

$$d^2 = \sum_{i \in \{x, y, z\}} (p_i - c_i)^2 \leq r^2 \iff \text{Collision Detected} \quad (40)$$

6.3 Progressive Multi-Stage Fracture Physics

Rather than simplistic binary deletions, the fortress responds realistically to artillery bombardment:

- **Curtain Wall Segments:** Stone curtain walls are discretized into modular wall segments. Each segment has an internal health pool (100%). Each cannonball hit deducts 10% health, darkening the segment's diffuse color override. Upon reaching zero health, the segment fractures, dropping its solid collision box so subsequent cannonballs fly freely through the breached breach.
- **Reinforced Wooden Gates:** The castle doors consist of two thick timber panels with iron strapping. A single hit damages the wood planks; two direct hits shatter the door completely into splinters, opening the central passageway.
- **Tower Destruction:** Flanking gate towers absorb up to three direct artillery impacts before collapsing.

7 Autonomous Battlefield Simulation and Character Animation

7.1 The 6-Phase Tactical Siege State Machine

Pressing key **B** initiates a fully autonomous, scripted tactical siege simulation executed by an internal state machine (Figure 8):

1. **Phase 0: Inactive / Peaceful Patrol:** Castle wall sentries patrol the north and south curtain walls. Relief guards periodically climb wooden siege ladders to replace sentries. Cannons and army remain parked in staging positions.
2. **Phase 1: Advance & Alarm:** The alarm rings. Fortress defenders immediately crank the winches to raise the heavy drawbridge ($0^\circ \rightarrow 85^\circ$). Simultaneously, the three field cannons and the entire army battalion march in synchronized lockstep from staging ($x = -24\text{m}$) to battle lines ($x = -13\text{m}$).
3. **Phase 2: Archer Barrage & Infantry Defense:** Castle archers loose coordinated volleys of arrows. The attacking infantry immediately raise their wooden tower shields to protect themselves. **Cannoneer 1** steps out unshielded to prime the fuse on Cannon 1, is struck by arrow fire, and falls in action. Cannon 1 is disabled. An infantryman breaks rank from the army line, rushes across to relieve Cannoneer 1, and restores Cannon 1 to operational status.
4. **Phase 3: Drawbridge Hinge Severing:** Cannoneer 2 (preserved unharmed at the center cannon) fires a direct precision shot at the drawbridge suspension chains. The chains snap, dropping the heavy timber bridge flat across the moat.
5. **Phase 4: Coordinated Artillery Bombardment:** All three cannons aim and unleash simultaneous cannonades: Cannon 1 shells the left curtain wall, Cannon 3 pounds the right curtain wall, and Cannon 2 fires directly into the wooden fortress doors, shattering them into kindling.
6. **Phase 5: Courtyard Breach & Melee Victory:** With all front defenses broken, the attacking infantry lower their shields, form into a disciplined two-file marching column (the maximum width the drawbridge accommodates), march across the bridge, and charge through the broken

archway into the fortress courtyard. Swords clash in animated melee combat until the fortress falls and the golden courtyard crest pulses in victory.



(a) Barrage: Shields Raised & Cannoner 1 Hit



(b) Courtyard Melee after Gatehouse Breach

Figure 8: Battlefield simulation milestones captured during real-time execution.

7.2 Procedural Character Kinematics and Animation

Every character (soldiers, archers, cannoners) is built from procedural geometric primitives with articulated hierarchical joints:

- **Sinusoidal Marching Kinematics:** Legs and arms swing in natural counter-phase:

$$\theta_{\text{left_leg}}(t) = A_{\text{walk}} \sin(\omega t), \quad \theta_{\text{right_leg}}(t) = -A_{\text{walk}} \sin(\omega t) \quad (41)$$

where $\omega = 7.5 \text{ rad/s}$ and $A_{\text{walk}} = 28^\circ$.

- **Shield Guard Animation:** When taking fire, soldiers raise their left arm and shield across their chest ($R_x(65^\circ)$), forming an impenetrable shield wall.
- **Archer Volley Sequence:** Archers execute a three-stage draw cycle: nock arrow, draw bowstring back, release with recoil.
- **Death Fall Kinematics:** Upon fatal hit, character models pitch backward onto the ground ($R_x(90^\circ)$) with spark particle bursts.

8 User Interaction, Controls, and Parameter Tuning

8.1 Comprehensive Keyboard and Mouse Control Scheme

The application provides complete interactive control over every visual and physical parameter, summarized in Table 3.

Key / Input Binding	Action / Functional Description
Mouse Left Drag	Rotate camera view (spherical orbit yaw and pitch)
Mouse Wheel	Zoom camera in / out (12.0 m to 240.0 m radius)
→ / ←	Drive active cannon forward / backward along ground (wheels roll with no slip)
↑ / ↓	Elevate / depress active cannon barrel (0° to 45°)
W / S	Steer / yaw active cannon barrel left / right
Space	Fire active cannon (spawns ballistic ball, barrel recoils, emits muzzle flash sparks)
1, 2, 3	Select Cannon 1 (left), Cannon 2 (center), or Cannon 3 (right) individually
A	Select all 3 cannons simultaneously for synchronized artillery salvo
G	Toggle Shading Mode: Switch between Gouraud (per-vertex) and Phong (per-fragment)
X or Y	Toggle Soft Shadow Mapping: Enable / disable 3×3 PCF light-space depth shadows
K	Toggle Real-Time Ray Tracing: Enable analytical ray-box slab shadow rays
R (Hold)	Manually raise drawbridge ($0^\circ \rightarrow 85^\circ$); release to lower back down
N	Toggle Day / Night: Switch solar illumination with lunar palette and night sky
B	Start Automated Siege Simulation: Trigger 6-phase autonomous tactical battle
P	Pause / resume the active battle simulation
T	Restart / reset battle simulation back to peaceful staging positions
F11	Toggle between fullscreen and windowed display mode
Esc	Exit application

Table 3: Comprehensive interactive keyboard and mouse control reference.

8.2 Parameter Adjustment Guide for Evaluation

To assist the evaluator in verifying code comprehension and dynamic adaptability, key parameters can be modified live in source files:

- **Projectile Velocity:** Modify initial launch velocity in `Main.cpp` line 523 (default $v_0 = 35.0$ m/s). Increasing to 50 m/s flattens the trajectory.
- **Gravitational Acceleration:** Modify `Projectile.cpp` (default $g = -9.81$ m/s²). Setting to -4.0 m/s² demonstrates low-gravity lunar ballistics.
- **Light Direction and Color:** Adjust `lightDir` in `Main.cpp` line 586. Modifying `dirLightColor` in `lit.frag` alters atmospheric warmth.
- **Shadow PCF Kernel:** Adjust loop bounds in `lit.frag` line 74 from $\{-1, 1\}$ to $\{-2, 2\}$ for a wider 5×5 ultra-soft penumbra.
- **Material Specular Exponent:** Modify `shininess` in `lit.frag` line 158 (e.g. increase metal from 96.0 to 256.0 for mirror-like highlights).

9 Conclusion and Academic References

9.1 Summary of Accomplishments

The completed project represents a rigorous, production-grade Computer Graphics demonstration built from mathematical foundations up:

1. Complete fulfillment of all proposal specifications with zero incomplete placeholders.
2. Exact mathematical formulations for 3-D affine transforms, kinematic wheel constraints, and trunnion elevation pivots.
3. Dual-mode shading implementation enabling live side-by-side comparison of Gouraud versus Blinn–Phong lighting models.
4. Multi-light architecture with directional sun/moon, hemispherical ambient bounce, and distance-attenuated point sources.
5. Dual shadowing implementations: 3×3 PCF soft depth mapping and GPU ray-box slab ray tracing.

6. Stable semi-implicit Euler integration for ballistics and continuous sphere-vs-AABB collision detection.
7. Rich procedural materials, atmospheric distance fog, and ACES filmic tone mapping.
8. An autonomous, multi-phase medieval battlefield simulation demonstrating dynamic character animation and progressive fortress destruction.

9.2 References and Sources

1. de Vries, Joey. *LearnOpenGL: Learn Modern OpenGL Graphics Programming in C++*. Packt Publishing, 2020.
2. Akenine-Möller, Tomas, Eric Haines, Naty Hoffman, Angelo Pesce, Michal Iwanicki, and Sébastien Hillaire. *Real-Time Rendering, Fourth Edition*. CRC Press, 2018.
3. Blinn, James F. “Models of light reflection for computer synthesized pictures.” *ACM SIGGRAPH Computer Graphics*, 11(2):192–198, 1977.
4. Gouraud, Henri. “Continuous shading of curved surfaces.” *IEEE Transactions on Computers*, C-20(6):623–629, 1971.
5. Phong, Bui Tuong. “Illumination for computer generated pictures.” *Communications of the ACM*, 18(6):311–317, 1975.
6. Kay, Timothy L., and James T. Kajiya. “Ray tracing complex scenes.” *ACM SIGGRAPH Computer Graphics*, 20(4):269–278, 1986.
7. Williams, Lance. “Casting curved shadows on curved surfaces.” *ACM SIGGRAPH Computer Graphics*, 12(3):270–274, 1978.
8. OpenGL Architecture Review Board. *OpenGL 3.3 Core Profile Specification*. The Khronos Group, 2010.